Donghyung Lee

DevOps Engineer | Resume & Career Description · 4Y 3M Experience

E-mail: genius5711@gmail.com Blog(KR): somaz.tistory.com Blog(EN): somaz.blog GitHub: github.com/somaz94

Introduction

Hello, I am Donghyung Lee, a DevOps Engineer.

Aiming for stable infrastructure operation, I codify every change as IaC and turn incident response into postmortem-driven assets to cut cloud costs 20-50% while raising operational efficiency. My core strength is interrogating the purpose of each task and pursuing a better direction.

I design and build infrastructure across AWS and GCP cloud and an OpenStack-based on-premises PaaS. I standardize container operations with Kubernetes, codify infrastructure with Terraform and Ansible, and run GitLab CI / ArgoCD-centered deploy pipelines alongside monitoring and logging stacks. Repetitive work is automated in Python and Bash.

I also operate adjacent data-engineering and AI/ML ops through data-pipeline automation and an in-house GPU LLM service, and build in-house DevOps tools myself — git-bridge, a Slack APK bot, and more.

Across 3 companies I solely led AWS → GCP migration, AWS + on-premises hybrid, and on-premises PaaS builds — achieving 20% AWS and 50% GCP cost cuts, 50-67% faster deploys, and 90%+ manual-work automation. Most recently I built an AWS EKS multi-cluster for a new game's cloud production launch, migrated helmfile-push deploys to a more stable auto-syncing model (ArgoCD pull-GitOps), and — alongside resolving 14-second game-DB stalls and unifying login via Keycloak central SSO — raised operational efficiency another notch.

I aim to keep contributing to infrastructure architecture advancement and organizational productivity.

Strengths

Asking 'Why' First
- Habit of asking 'why' first — define the business problem before picking the tool

Systematic Documentation & Knowledge Sharing
- Document and share every task, building organizational knowledge assets

Record-Based Continuous Improvement
- Postmortem-record every incident, infra change, and troubleshooting — preventing recurrence, driving improvement

Full-Stack Infrastructure Experience
- End-to-end infra across cloud (AWS, GCP), on-premises, networking, CI/CD, and monitoring

Skills

Cloud	AWS, GCP, OpenStack
Container Orchestration	Kubernetes, Docker, Cilium CNI, NGINX Gateway Fabric / Gateway API, Karpenter
Infrastructure as Code	Terraform, Ansible, Helm, Helmfile
CI/CD	GitHub Actions, GitLab CI/CD, ArgoCD, ArgoCD ApplicationSet, Jenkins, GitOps
Monitoring & Logging	PLG Stack (Prometheus/Loki/Grafana), ELK Stack, EFK Stack, ECK Operator, Fluent Bit, Thanos
Security & IAM	Keycloak (Operator, OIDC IdP), Vaultwarden, GitLab SSO (OIDC), External Secrets Operator, Sealed Secrets
Automation	Go, Python, Shell Scripting
Database	MySQL, PostgreSQL, Redis, MongoDB
OS & Servers	Ubuntu, Rocky Linux, CentOS, Debian, GPU Server

Core Competencies

Cloud Infrastructure Design & Operations

Multi/hybrid cloud architecture design and operations experience on AWS and GCP

- AWS EKS, GCP GKE production operations & cost optimization (20% reduction via hybrid transition, 50% via GCP migration, savings reinvested in new projects)
- IaC-based infrastructure automation and version control using Terraform (100% GCP resource codification)
- On-premises to cloud hybrid architecture design and migration experience
- Kubernetes storage performance optimization (control-plane HDD → SSD migration — etcd fsync 1760ms → 3.88ms = 454× faster; GitLab Runner build I/O isolation recovered game DB COMMIT latency from 14s back to ms)

CI/CD Pipeline & GitOps

Maximizing development productivity through GitOps-based deployment automation

- CI/CD pipeline design and enhancement using GitHub Actions, GitLab CI, ArgoCD
- 50-67% deployment time reduction with continuous GitOps deployment shortening Time to Market, 95% rollback time decrease (30min → 1-2min)
- Jenkins-based Unity Android/iOS build automation and Slack-integrated deployment notifications
- Built a layered shared toolchain image architecture + auto version cycle standardizing 15 consumer repos — first-job install time 5min → ~10s (~95% reduction), with 4-stage cascade automated by just 2 manual clicks

Monitoring & Observability System Implementation

Establishing proactive incident detection and rapid response systems

- Integrated metric/log monitoring with PLG Stack (Prometheus, Loki, Grafana)
- Evolved from ELK → EFK Stack, then redesigned into ECK Operator-based Elastic Stack 9.0 CRD declarative management (automatic TLS rotation, automated rolling upgrades)
- Reduced MTTR and improved service availability via SLI/SLO-based alerting
- kube-prometheus-stack with custom alert rules, Grafana dashboards, and 4 DB exporters (MySQL, Redis, Elasticsearch, PostgreSQL) integrated
- Fluent Bit data integrity & HA hardening (SQLite offset DB + PVC, PodDisruptionBudget + preStop hook, 7 Prometheus alerts — zero log gap / duplication on Pod restart)

Automation & Efficiency

Operations automation and developer productivity tools using Python and Bash

- 90%+ manual work reduction through data collection, deployment, and document management automation
- Internal LLM service (Ollama + Open WebUI) ensuring 100% internal security guideline compliance and dev team productivity improvement
- Internal tool development: GitLab Webhook doc sync, Slack Bot APK deployment notifications
- Migrated shell → Python (stdlib only, 758 unittests) + wave-based auto-deploy across 25 components + auto-upgrade MR generation
- Established self-sufficient Kubernetes ops at DP (Delivery Partner) sites — designed and ran a 6-month training program (avg 10 attendees, 4h/session), standardized materials reused across other clients

Kubernetes Platform Modernization & Open-Source Contribution

Zero-downtime migration of network/logging stacks and releasing reusable public charts as open source

- Migrated multiple ingress-nginx instances to a single NGINX Gateway Fabric control-plane + per-class Gateway CRs with zero downtime (full cutover, wildcard TLS consolidated)
- Migrated the ECK Operator-based logging stack from a local chart to an OCI chart with zero downtime (CR/PVC names preserved, cluster_uuid unchanged, no data loss)
- Released 3 open-source Helm charts (nginx-gateway-cr, elasticsearch-eck, kibana-eck) and multiple composite GitHub Actions on ArtifactHub / GitHub Marketplace

Infrastructure Security & IAM

Centralized in-house authentication and secret governance via Keycloak Operator-based SSO and Vaultwarden password management

- Introduced central SSO with the Keycloak Operator + KeycloakRealmImport — let existing GitLab accounts keep logging in, integrated ArgoCD / Harbor / Vaultwarden OIDC (zero downtime, verification checks PASS), and established a single authorization flow from GitLab groups → Keycloak mappers → ArgoCD roles
- Deployed Vaultwarden on Kubernetes via Helm with GitLab SSO + automated backups (SQLite dumps, 30-day retention) + interactive restore script — centralized management for 70 users and 50 secrets
- git-bridge Retry API — zero webhook-secret exposure + SRE-friendly recovery (Bearer auth, repo-level direction override)

Data Engineering & Internal LLM Services

End-to-end ownership from GCP data pipelines to in-house LLM services

- Built a multi-source (MongoDB · CloudSQL · GA · Dune) → BigQuery → Google Sheets pipeline by combining BigQuery, Dataflow, Cloud Functions, and Cloud Scheduler — entire infrastructure 100% Terraform IaC
- Deployed an internal LLM service (Ollama + Open WebUI) on a Kubernetes-hosted on-premises GPU server, saving ~$100/month by replacing external AI SaaS subscriptions and ensuring 100% internal security compliance

- 95% data-collection automation and 0% data gap rate by eliminating human error

---

Professional Experience (Detailed)

Phantom (Concrit Studio)

2024.10 ~ Present

DevOps Engineer

As the sole DevOps infrastructure engineer at a game development company, I design, build, and operate AWS-based cloud and on-premises servers. Development and test servers are on-premises, while QA and Production servers are on AWS, operating a hybrid architecture that reduced monthly infrastructure costs by 20% ($60K → $48K) and channeled the savings into new project investments.
Currently providing secondary technical support and operations for a live-service game in collaboration with the external publisher, and building on-premises/AWS infrastructure to improve development productivity for new game projects.

Project 1: AWS-On-premises Hybrid Cloud Architecture

Contribution 100% (Solo design & implementation, in collaboration with the external publisher's ops team) 2024.10 ~ Present (In Operation)

Problem

While operating the entire infrastructure on AWS, high cloud costs of $60,000/month necessitated cost optimization.

Approach

Workload-specific cost analysis revealed that dev/test environments had minimal traffic variation, making cloud auto-scaling unnecessary, while only QA/Production required scalability and stability. Designed a hybrid architecture transitioning dev/test to on-premises while keeping QA/Production on AWS.
The two environments are operationally independent without VPN (application traffic isolated); the image registry is also split by environment (dev = Harbor, QA/Production = ECR), with each environment building independently to match its architecture. Unified operational standards through Terraform and Ansible IaC integration (100% IaC coverage for new components, IAM excluded) across both environments, achieving consistent infrastructure management without increasing operational complexity.

Troubleshooting

- When configuring ALB Ingress on EKS, needed to route to different backends per game client version.
  Resolved by combining custom header-based routing rules with Target Groups in ALB conditional routing
- Intermittent timeouts during ClusterIP service communication in Kubernetes IPVS mode. Resolved through IPVS conntrack threshold tuning
- During rolling deploys of the ArgoCD-managed production services, the ALB target deregistration delay didn't line up with Kubernetes Pod SIGTERM handling, causing some session connection drops. Solved via 3 changes — achieving zero-downtime deploys with 0 connection drops:
  · (1) Aligned ALB drain timing — shortened the target group deregistration_delay from the default 300s to 30s and added a sleep 35s in the Pod lifecycle.preStop hook. During termination the readiness probe fails first, so ALB marks the target unhealthy and stops new requests while in-flight requests keep being served (zero downtime). terminationGracePeriodSeconds is set to exceed preStop (35s) — for example, services whose app self-drains in-flight requests on SIGTERM (~30s) get 75s (preStop 35s plus drain headroom), others 60s
  · (2) Tuned the RollingUpdate strategy — pinned maxSurge to an absolute 1 (a percentage would spawn many nodes at once, so it's controlled by absolute value), with maxUnavailable set to fit replica scale — for example, replica 2~3 uses maxUnavailable 1 to keep ~100% capacity during surge-first rollout, replica 1 uses "50%"→0 rounded for surge-first zero-downtime, and large counts (replica 10~20) use maxSurge 0 + maxUnavailable 20~25% for batch rollout without spinning up new nodes
  · (3) Applied a PDB to multi-replica (2+) services — protecting them so Karpenter node consolidation evicts only one Pod at a time (single-replica services have no PDB)

Results

- Workload analysis-based infra optimization reducing monthly costs by 20% ($60,000 → $48,000), reinvesting ~$144,000 annual savings into new project infrastructure for resource reinvestment cycle
- Offloaded dev servers to on-premises, cutting ~$1,000/month in AWS resource costs and optimizing resource usage
- Ensured operational stability and fault isolation through environment role separation

Tech Stack

AWS (EKS, EC2, ALB, Route53, ACM, EFS, Aurora MySQL, ElastiCache, CloudFront), Kubernetes, Terraform, Helm

---

Project 2: On-premises CI/CD Pipeline Construction & Enhancement

Contribution 100% (Full infra & pipeline ownership) 2024.12 ~ Present (In Operation)

Problem

Developers manually built and deployed to servers, taking an average of 30 minutes per deployment with frequent failures due to human errors.

Approach

Built a GitOps-based automated deployment environment combining GitLab CI and ArgoCD.
Built Docker images with Kaniko without Docker-in-Docker (no privileged container), and configured multi-environment (alpha/review/dev/staging) deployments in a single pipeline.

Troubleshooting

- Applying `selfHeal=true` uniformly risked prod changes auto-applying without human approval → split by environment: prod uses `selfHeal=false` + manual sync windows to gate deploys, dev/staging keep `selfHeal=true` for automatic drift correction, with every sync event wired to Slack for change traceability, stability, and emergency-response flexibility
- Package installation failure on CI Runner due to GPG key expiration after GitLab upgrade. Standardized the apt-key renewal procedure into a runbook for fast recovery on recurrence
- Build time spikes due to cache invalidation during Kaniko builds. Resolved with `--cache=true --cache-ttl=24h --snapshot-mode=redo` option combination
- Incrementally tuned the data-deployment pipeline's source-fetch bottleneck — the prior `git checkout + git pull` re-fetched the full history (~3.5GB) each run (~139s); replaced with `git fetch --depth=1 --filter=blob:none` (fetch only the tip commit + file contents lazily, minimizing transfer), cutting source-fetch from ~139s to ~30s
- Optimized npm install with `npm ci --cache .npm --prefer-offline` + lockfile-hash cache key to reuse tarball downloads (invalidated only on dependency change); moved data upload from direct SSH scp/rsync to S3 transit-bucket staging → SSM SendCommand-based `aws s3 sync --exact-timestamps`, removing SSH keys and deploy-account key-leak risk

Results

- 67% deployment time reduction (30min → 10min), 90% manual work automation
- Split deployment via GitOps automation — dev/qa/review run continuous merge-driven deployment (~100/week per core service repo) to cut new-feature Time to Market, while prod ships controlled ~5-7 releases/week gated behind manual sync windows for stability, and deploy-resource efficiency lets the dev team focus on feature development
- Cross-repo MasterData pipeline orchestration — a data-repo change auto-fans-out to downstream services (some to all envs, others limited to dev/qa); the receiving pipeline commits only on real changes before triggering its own build, gated on trigger source/type to prevent circular (loop) triggering

Tech Stack

GitLab CI/CD, ArgoCD, Jenkins, Kaniko, Kubernetes, Docker, Harbor

---

Project 3: Central Logging & Observability Platform (Collection → Integrity/HA → Product Analytics)

Contribution 100% (Sole design across the full lifecycle — collection · integrity · product analytics) 2024.11 ~ Present (~18 months total)

Starting from a log-scattered on-premises environment, advanced centralized logging through three stages (ELK → EFK → ECK Operator), hardened collector data integrity / HA, then repurposed the accumulated EFK log pipeline into a game user-retention analytics platform. Solely designed and operated the full observability lifecycle — build → operate → improve → analytics.

3-1. Centralized Logging System (ELK → EFK → ECK Operator 3-stage Enhancement, 2024.11 ~ Present)

With server/container logs scattered across the on-premises environment — slow incident root-cause and no unified monitoring — advanced centralized logging in 3 stages.

Phase 1 (ELK): built with Elasticsearch + Logstash + Kibana + Filebeat — Filebeat collects → Logstash JSON parsing/field mapping → Elasticsearch storage → real-time monitoring/search via Kibana dashboards.

Phase 2 (EFK migration): migrated to Fluent Bit + Fluentd to relieve Logstash JVM memory (1GB+), Filebeat's Helm-ecosystem mismatch, and custom-extensibility limits. A lightweight C-based Fluent Bit DaemonSet collects with automatic K8s metadata tagging while Fluentd handles multi-output processing/filtering before forwarding to Elasticsearch; a chart-upgrade automation script auto-preserves custom PVC templates across upstream chart upgrades.

Phase 3 (ECK Operator + Stack 9.0): with Elastic's official Helm Chart stalled 2+ years and CRD-based declarative management becoming mainstream, performed a Stack 8.5.1 → 9.0.0 major upgrade + deployed ECK Operator 3.3.2 in the elastic-system ns. Redefined ES/Kibana as Custom Resources (CR wrapper local chart), zero-downtime cutover after parallel monitoring → logging ns deployment, and established automatic TLS rotation (1y/30d), Helm-templated elastic password, and single-line CR spec.version rolling upgrades.

Phase 4 (AWS(EKS) logging expansion): templated the on-premises EFK stack into five AWS(EKS) `-aws` stacks (elasticsearch-aws / kibana-aws / fluent-bit-aws / fluentd-aws / eck-operator-aws), porting the same logging model onto AWS, with ILM hot/warm/cold 60-day delete + SLM 90-day S3 snapshot retention automated over IRSA (no static keys).

Troubleshooting: Logstash null-byte (\x00) parse breakage (mutate gsub preprocessing), field mapping conflict mapper_parsing_exception (explicit index-template mapping + type conversion), disk-watermark read-only (ILM 7-day auto-delete), [EFK] Fluent Bit NFS WAL write failure (custom PVC patch auto-preserve) & Fluentd buffer overflow log loss (chunk_limit_size/flush_interval tuning + overflow_action=block), [ECK] Kibana 9 secure-cookie HTTP login block (tls.disabled + publicBaseUrl http:// + secureCookies:false), no Fluentd ES9 variant (fluent-plugin-elasticsearch 5.4.4 suppress_type_name:true for ES 9.0 compat on ES8 image + active-marker E2E verify), 3.3.x stackconfigpolicy-controller GET-ing resources outside managedNamespaces every 17 min (managedNamespaces:[] cluster-wide watch workaround).

Results: 90% log-search time reduction & improved MTTR, EFK migration removed Logstash JVM (1GB+) burden, 1GB/day real-time processing with 90-day retention, ECK migration removed manual TLS/password/StatefulSet upgrade burden + Stack 9.0 major upgrade closed a 2-year gap, Operator secureMode + ServiceMonitor auto-registers controller-runtime metrics into kube-prometheus-stack, and templating five AWS(EKS) `-aws` stacks ported the same logging model onto AWS (ILM 60-day delete, S3 SLM 90-day, over IRSA).

Tech Stack

Elasticsearch, Fluent Bit, Fluentd, Logstash, Kibana, Filebeat, ECK Operator, CRD, Custom Resource, Kubernetes, Helm

---

3-2. Fluent Bit Log Collector Data Integrity & HA Hardening (2026.03 ~ 2026.05)

Post-Phase-3 ECK operation revealed that whenever a Fluent Bit Pod restarted its in-memory tail offsets were lost (re-index/loss risk), and with a single replica and no PDB/preStop hook, node drain caused collection gaps while no collector-level alerts left room for silent failure.

Tier 1 (data integrity): introduced a SQLite offset DB (`db /var/log/flb-storage/tail.db`) + dedicated 2Gi PVC so offsets survive Pod restarts — verified zero re-index / zero duplicates on helmfile revision 4.

Tier 2 (availability): added PodDisruptionBudget (`minAvailable=1`) + preStop hook (`sleep + flush`) + liveness/readiness probes (/api/v1/health) to minimize collection gaps during node drain / rolling updates.

Tier 3 (monitoring): 7 PrometheusRule alerts (input/output failure rates, retry buildup, fluentd queue pressure, fluent-bit Pod down, SQLite DB write failures, etc.) + Slack alert routing surface silent failures and hand off to operators.

Tier 4: documented 3 HA expansion options (input-path partitioning / sidecar / Vector multi-collector).

Results: zero re-index / zero duplicates on Pod restart guaranteeing log integrity, PDB+preStop minimizing collection gaps (direct MTTR contribution), self-monitoring on the observability stack itself, and 3 documented HA options reducing future decision cost.

Tech Stack

Fluent Bit (tail / SQLite offset), Kubernetes PVC / StorageClass, PodDisruptionBudget, Prometheus Operator (PrometheusRule), Helmfile, Fluentd (buffer tuning)

3-3. Game User Retention Analytics Platform (EFK Logs → Product Analytics, 2026.06 ~ Present)

The EFK stack served only incident response / log search with no product-analytics metrics for the game's new users, retention, and churn, so we reused the existing log pipeline without a separate SaaS. On top of EFK, an Elasticsearch Transform (continuous, 5-minute) pivots raw events into a per-user cohort index (first_seen / active_dates / max_cleared_chapter), with a KST runtime field correcting day boundaries. On that index we built a 12-panel Kibana dashboard (NU/DAU/WAU/MAU KPIs, NU·DAU trends, D+1~D+30 retention curves, daily cohort-retention table, chapter distribution) and documented a per-environment templating guide for the dashboard and Transform definitions.

Results: extended an incident-focused log stack into a product-analytics platform — quantifying new users, retention, and chapter progression without a separate SaaS, delivering D+1~D+30 retention curves and a daily cohort table, packaged as reusable per-environment templates.

Tech Stack

Elasticsearch Transform, Kibana, EFK Stack, Runtime Field (KST), ECK Operator, Kubernetes

---

Project 4: Developer Productivity Tools (APK Deploy Bot · Doc Automation · LLM Service · Git Mirroring · Static File Server)

Contribution 100% 2025.02 ~ Present

4-1. Slack APK Distribution Automation Bot (4 weeks, 2025.02)

Auto-notification to Slack with QR code on Jenkins build completion. 90% QA team delivery time reduction

Tech Stack

Python, Slack Bot API, Jenkins, Kubernetes, Docker

4-2. GitLab-Google Drive Document Automation System (2 weeks, 2025.06)

Auto-sync on push via GitLab Webhook + Google Drive API. 80% document management time reduction.
Incrementally optimized the upload job with rclone `--checksum` to skip unchanged files + sparse-clone to clone only the target directory

Tech Stack

GitLab Webhook, Google Drive API, Python, Bash Script

4-3. Internal LLM Service Deployment (4 weeks, 2025.11)

Deployed Ollama + Open WebUI on Kubernetes on on-premises GPU server. Serves 15 developers with avg 100 queries/day, with an authored user guide and personally driven onboarding cementing adoption.
Saved ~$100/month by replacing external AI SaaS subscriptions. 100% internal security guideline compliance and eliminated data leakage risks

Tech Stack

Ollama, Open WebUI, Kubernetes, NFS, GPU Server

4-4. Git Repository Mirroring Tool + Retry API Follow-up (4-week initial build, 2026.02 ~ 2026.06)

Developed a Go-based bidirectional Git mirroring tool (git-bridge). Ensured code quality with unit tests and a CI/CD pipeline (GitLab CI for development, GitHub Actions on the public OSS mirror), fully automating sync across CodeCommit/GitLab/GitHub for 10 repos with avg 30 sync events/day.
Phase 2 (2026.05) added a `POST /retry/mirror` endpoint with Bearer-token auth (`RETRY_API_TOKEN`, separated from the webhook secret, constant-time compared) — a safe manual-retry path that no longer exposes the webhook secret. Direction supports `auto / source-to-target / target-to-source`, with per-repo `retry_direction` overrides and per-repo Slack webhook env vars for split notifications. Added 11 new tests, passing the CI test gate
Phase 3 (2026.06) root-caused a non-fast-forward regression where multiple uncoordinated force-pushes rolled a shared branch back to a stale snapshot, and designed `ref_overrides` to pin the sync direction per ref pattern. The repo stays bidirectional while specific branches are pinned one-way, preventing reverse-direction echoes from rolling a branch back; scoped pushes to the triggered ref (removed `--all`, applied only to override repos so existing repos behave exactly as before), added a direction restriction on delete events, and a fail-open fallback. Verified 6/6 runtime checks on a test-only repo (production repo isolated and unaffected), landing 4 commits (feat / fix / docs / chore) on main

Tech Stack

Go, AWS SQS, GitLab Webhook, GitHub Webhook, HTTP Bearer Auth, Slack Webhook, Kubernetes, Docker

---

4-5. Static File Server - In-house Development (Open Source, 2026.03 ~ Present)

Developed a lightweight Go-based static file server (static-file-server) in-house and operated a Helm repository on GitHub Pages. Features: dark-mode directory listing UI, file preview (image/video/PDF/text), search with URL-hash sharing, multi-select + ZIP batch download, Gzip, Prometheus metrics, JSON logging, automatic APK Content-Type.
Deployed to Kubernetes via an in-house Helm chart (NFS storage), fully replacing the legacy halverneus/static-file-server. Handles ~30 downloads/day and 5 APK builds/week (daily build). Released as open source under Apache-2.0.

Tech Stack

Go, Helm Chart, GitHub Pages, Docker Hub, Kubernetes, NFS, Prometheus

---

Project 5: Kubernetes Cluster Operations Enhancement

Contribution 100% (Solo design — monitoring · upgrade automation · Helm management framework · NGF migration · 3 OSS Helm charts · storage performance optimization · Shell→Python migration; captured as 4 automation scripts establishing a standard procedure → so the team can reuse)
2025.12 ~ Present

To advance cluster visibility, operational stability, network modernization, storage performance, and automation-code quality, overhauled monitoring, automated K8s upgrades and Helm management, and migrated the network controller onto the Gateway API.
Released the artifacts from the NGF migration and ECK Operator adoption as OSS Helm charts, resolved storage bottlenecks via control-plane / DB-node SSD migration and build I/O isolation, and modernized the infra deploy/upgrade pipeline with wave-based automation + a Shell → Python migration.

5-1. Monitoring & Alerting Enhancement (2025.12 ~ Present)

Designed custom alert rules and Grafana dashboards based on kube-prometheus-stack and integrated MySQL/Redis/Elasticsearch/PostgreSQL exporters. Deployed node-exporter on physical servers and VMs via Ansible, and collected Cilium CNI agent/operator metrics via kubernetes_sd_configs. Configured severity-based Slack routing with inhibit rules in Alertmanager to improve alert quality.
(Chose kube-prometheus-stack over SaaS Observability — Datadog / New Relic — because on-premises metric sovereignty, exporter freedom, and zero runtime cost mattered more.)

Tech Stack

Prometheus, Grafana, Alertmanager, Cilium, node-exporter, Ansible, Slack API

5-2. K8s Upgrade Automation & Helm Chart Management Framework (2025.12 ~ Present)

Kubespray upgrade automation script for pre-flight, backup, compatibility checks + multi-step health check script reducing verification time by 90%.
Upgrade framework across the Helm charts + 4 canonical templates synchronizer (check for CI drift detection, apply for batch propagation) maintaining script body consistency.
Automated Kubernetes certificate renewal script + crontab (across nodes, every 6 months at 3 AM) proactively preventing API Server outages from certificate expiration
(Kept Kubespray — kOps is AWS-only, Cluster API's on-premises bootstrap is heavyweight; Kubespray fits the Ansible-familiar ops, so only added the automation layer.)

Tech Stack

Kubespray, Ansible, Helm, etcd, Shell Script, GitLab CI

5-3. Ingress-nginx → NGINX Gateway Fabric (NGF) Migration (2026.04)

Migrated multiple ingress-nginx controller instances (default + public-a~j, each with a pinned MetalLB LoadBalancer IP) to a single NGF 2.x control-plane + per-class Gateway CRs, preserving MetalLB IPs so DNS/firewall stayed unchanged via parallel deployment and gradual cutover (30-60s actual downtime per class).
Executed Phase 0~7+ (MetalLB pool expansion → NGF install → observability apps → HTTP-only / ApplicationSet apps → HTTPS-enforced apps (Harbor, Vaultwarden) → IP-swap cutover → Ingress cleanup). Mapped ingress-nginx annotations 1:1 to Gateway API + NGF CRDs (HTTPRoute filters, ClientSettingsPolicy, ProxySettingsPolicy, RateLimitPolicy, BackendTLSPolicy) and unified the HTTPRoute values schema across the charts for batch migration of ApplicationSet apps.
Consolidated per-app self-signed TLS Secrets into a single wildcard TLS Secret (10-year validity), cutting manual renewal overhead by 50%.

Tech Stack

NGINX Gateway Fabric, Gateway API, HTTPRoute, ClientSettingsPolicy, ProxySettingsPolicy, RateLimitPolicy, BackendTLSPolicy, Helmfile, MetalLB, ArgoCD

5-4. Three Open-Source Helm Charts — nginx-gateway-cr · elasticsearch-eck · kibana-eck (2026.04 ~ 2026.05)

nginx-gateway-cr: refined the local CR chart extracted from the NGF migration (5-3) into a reusable chart that deploys five tenant-level resources — Gateway / NginxProxy / ReferenceGrant / ServiceMonitor / PodMonitor. Includes a multi-Gateway-friendly schema (`gateways[]` array with per-gateway overrides) and PodMonitor templates recommended for NGF 2.x (controller + dataplane).

elasticsearch-eck / kibana-eck: elasticsearch-eck (11 ES templates) + kibana-eck (10 Kibana templates) deploy the CRs, Secrets, HTTPRoute, Ingress, BackendTLSPolicy, ServiceMonitor, and NetworkPolicy from a single set of values. Added `values.schema.json` (draft-07) validation, Minimal/HA presets, and per-environment storageClass mapping in the README.

Unified distribution: all three charts ship simultaneously across OCI (ghcr.io/somaz94/charts) + a gh-pages Helm repo + ArtifactHub (networking / monitoring-logging, Apache-2.0). Added a new canonical version-tracking template.

Zero-downtime cutover: when migrating the mgmt cluster from the local chart to the OCI chart, every CR / PVC / Secret name was preserved — cluster_uuid stayed unchanged, with zero Elasticsearch pod restarts and a single Kibana rolling restart, i.e. no data loss. HA rolling scenarios passed 3/3 on a local kind cluster; end-to-end install on the real cluster reached Ready in 71s for Elasticsearch and 57s for Kibana.

Tech Stack

Helm Chart, Gateway API, NGINX Gateway Fabric, ECK Operator, Elasticsearch, Kibana, Prometheus Operator, OCI Registry, ArtifactHub, Kubernetes

5-5. Storage Performance Optimization — etcd / DB SSD Migration + Build I/O Isolation (2 weeks, 2026.04 ~ 2026.05)

The control-plane VM and the game-DB worker VM both ran on HDDs, contending with GitLab Runner buildx for I/O — etcd WAL fsync p99 ballooned to ~2s, apiserver pods GET p99 to ~7s, and game DB COMMIT latency to 9~14s, blocking play. Root cause pinned on the game-DB worker VM's saturated disk.

Step 1: tuned MySQL with 6 parameters (e.g. `innodb_flush_log_at_trx_commit=2`) for immediate relief.

Step 2 (control-plane VM SSD migration): sparse-copied the KVM qcow2 (11GB, 6m50s) + virsh XML edit, completing the cutover with 8.5 minutes of downtime (vs. a 15-minute estimate).

Step 3 (game-DB worker VM DB/Redis SSD migration): lossless migration of 35GB across game-db (MySQL) + 3-env valkey-redis via stop → backup → SSD migration → restore → local-path PV rebind.

Step 4 (build-only VM): provisioned a 4 vCPU / 16GB / 150GB SSD VM on a separate physical server via cloud-init, joined it with kubespray, and applied `node-role=build` label + `dedicated=build:NoSchedule` taint to isolate buildx. Captured the work as 4 automation scripts.

Postmortem & Recurrence Prevention: promoted the MAC sexagesimal pitfall into a MAC-match verification gate, promoted the etcd leader-election 5xx finding into an explicit 'etcd quorum recovery check' migration-runbook gate, and packaged the 4 automation scripts + config files as a standard procedure asset for future worker-node additions.

Results: **etcd WAL fsync 1760ms → 3.88ms (454×)**, **etcd backend commit 1810ms → 3.77ms (480×)**, **apiserver pods GET p99 6520ms → 23ms (283×)**, **game DB COMMIT 9~14s → ms range**, build-isolation VM validated 11 PASS / 0 FAIL

Tech Stack
KVM / libvirt, qcow2, virsh, etcd, Prometheus / PromQL, MySQL, Redis (valkey), kubespray, cloud-init, GitLab Runner

---

5-6. Infra Deploy/Upgrade Automation + Shell → Python Migration (2026.03 ~ Present)

The internal Kubernetes infra monorepo's 25 components were managed by helmfile + shell, hitting limits on multi-env branching and complex YAML patching. **Deploy automation**: structured the CI pipeline into 6 waves (bootstrap → core → security → db-redis → observability → apps) — MR diff detection → helmfile diff/apply on changed components only, with a manual gate making the wave-based deploy a standard. Added 10 new CI scripts (helmfile-changed-dirs, diff-component, apply-component, auto-upgrade, render-mr-body, …).

Upgrade automation: CI detects new versions across all 25 component charts and opens an MR with auto-rendered body (changed chart, values diff, breaking-change notes).

Shell → Python migration: rewrote 8 canonical templates + the sync and backup scripts + 6 CI scripts in Python (stdlib only). Built up 25 component-level upgrade modules + 758 unittests. Adopting the helmfile-tools image (from Project 7) brought **CI bootstrap time from 5min to ~10s (~95% reduction)**, with local (macOS venv) and CI container running the same code.
This helmfile-push deploy model was subsequently migrated to the ArgoCD pull-GitOps control plane in 5-7, replacing push-based deployment with pull-based declarative management (deploy-model evolution).

Tech Stack
Python 3.10+ (stdlib only), unittest, GitLab CI/CD, Helm / Helmfile, Git automation, yq

---

5-7. helmfile Push → ArgoCD Pull-GitOps Migration (App-of-apps Control Plane, 2026.06 ~ Present)

Platform releases deployed via helmfile push tied the deploy actor to the CI runner, and scaling to multiple clusters called for a pull-based GitOps control plane to declaratively converge per-cluster drift. A Matrix + Git-files generator ApplicationSet reads each component's `argocd/*.yaml` markers to emit Applications, registering 16 on-premises + 8 AWS = 24 platform releases across 2 clusters.

Zero-downtime adoption: adopted existing helm releases without re-creation (`resourceTrackingMethod=annotation`) and mapped helmfile deploy-waves 1:1 onto ArgoCD sync-waves to preserve deploy order.

App-of-apps bootstrap: bootstrapped as root → {app,infra}-projects (wave -1) → {app,infra}-appsets (wave 0), with prune-cascade guardrails keeping the control plane at `prune:false + selfHeal:true` and only leaves at `prune:true` to block mass deletion; upgrades follow an argocd-pin pattern.

Stabilization / least privilege: ArgoCD resource.exclusions removes control-plane-generated churn (Endpoints/EndpointSlice/coordination.k8s.io Lease/Authz·Authn) from tracking to trim watched events / UI clutter / controller load (applied on both on-premises and AWS), and tenant AppProject `clusterResourceWhitelist` / `namespaceResourceWhitelist` were tightened from

`*` to only chart-rendered kinds (cluster scope limited to Namespace·PersistentVolume, no ClusterRole/CRD, only infra at `*/*`) for least privilege and reduced blast radius.

Results: 24 releases across 2 clusters (16 on-premises + 8 AWS) declaratively managed from a single control plane, re-creation-free adoption for a zero-downtime cutover, prune-cascade guardrails eliminating GitOps mass-deletion risk, and resource.exclusions + least-privilege AppProject whitelists trimming UI clutter / controller load and shrinking blast radius (new kinds caught early via disallowed-resource sync failures).

Tech Stack

ArgoCD, ApplicationSet (Matrix / Git files generator), GitOps, Helm / Helmfile, Multi-cluster, Kubernetes

---

Project 6: Infrastructure Security System

Contribution 100% 2026.04 ~ Present

6-1. Vaultwarden Password Management System (1 week, 2026.04 ~ Present)

Deployed Vaultwarden on K8s via Helm, GitLab SSO integration, org/collection access control. Automated daily backup (30-day retention) + interactive restore.
Migrated 70 users and 50 secrets into centralized management

Tech Stack

Vaultwarden, Helm, OpenID Connect, GitLab SSO, Kubernetes

6-2. Keycloak Operator-based Central SSO Introduction (2026.04 ~ Present)

Replaced the layered setup where ArgoCD / Harbor / Vaultwarden each used GitLab directly as an OIDC IdP with a central broker (Keycloak). Introduced the Keycloak Operator + KeycloakCR + KeycloakRealmImport with a PostgreSQL backend, brokering GitLab as Identity Provider so existing accounts keep logging in. Codified Realm / Client / IdP / Group / Mapper via a `kcadm.sh` bootstrap script and migrated ArgoCD / Harbor / Vaultwarden's OIDC config from direct-GitLab to via-Keycloak. GitLab groups (`server` , `global-admin`) are brokered through Keycloak group mappers and wired straight into ArgoCD's role mappings, consolidating the authorization SSOT onto Keycloak.

Results: ArgoCD / Harbor / Vaultwarden OIDC integrations completed with zero downtime and verification checks PASS; captured five pitfalls hit during adoption (IdP providerId / scope / mapper config / Operator idempotency / Harbor user mapping) into the plan doc to prevent recurrence; laid the groundwork for future LDAP federation, MFA, and central session policy

Tech Stack

Keycloak Operator, KeycloakCR / KeycloakRealmImport, PostgreSQL, OIDC / OAuth2, kcadm.sh, ArgoCD, Harbor, Kubernetes Gateway API

6-3. GitOps Secret Management — External Secrets Operator · Sealed Secrets (2026.05 ~ Present)

Wired a ClusterSecretStore to AWS Secrets Manager via External Secrets Operator, converting plaintext DB-password values into ExternalSecret references (synced one sync-wave ahead of the Deployment). Encrypted the Harbor image-pull secret as a cluster-wide SealedSecret managed from an admin-only AppProject.
Routed ArgoCD notifications to separate Slack channels via notify-channel label routing.

Tech Stack

External Secrets Operator, AWS Secrets Manager, Sealed Secrets, ArgoCD, Helm, Kubernetes

---

Project 7: Shared Toolchain Image Layered Architecture + Auto Version Cycle

Contribution 100% (Architecture design · CI standardization · automation pipeline) 2025.09 ~ 2026.05

Problem

Across the 15 repos in the GitLab CI standardization scope (1 template provider + 14 consumers), each job re-installed helm / helmfile / kubectl / crane / aws-cli on top of an Alpine base, making first-job bootstrap take an average of ~5 minutes. Tool versions were scattered across the 14 consumer repos, making coordinated bumps effectively impossible.
Reliance on `:latest` tags also undermined reproducibility.

Approach

Layer 1 (Mirror): copied external registry images into the internal Harbor via `crane copy`, preserving multi-arch manifests.

Layer 2 (Custom): built 7 fat images on top of Layer 1 (helmfile-tools, node-build, go-build, rclone-backup, aws-cli-tools, kaniko-build, terraform-tools).

Tier 1~2 SSOT: introduced a shared `.toolchain_build_custom` template + central `TOOLCHAIN_*_TAG` variables, indirecting consumer repos away from tracking image tags themselves.

Shared CI template SSOT: extracted the CI jobs copy-pasted per repo into a central shared CI template repo, each repo referencing them via `include`. Manages the auth (keyless AWS auth) / build (kaniko) / deploy (ArgoCD) / backup (Google Drive) job templates plus shared anchors (git·ssh·rules·job-config·packages) from a single source, removing copy-paste drift.

Phase 4 auto cycle: a cron job drives the following 4 steps so the full cascade completes with just two manual clicks:

· (1) Upstream version detection
· (2) Layer 1 mirror auto-MR
· (3) Layer 2 rebuild + Tier 2 sync auto-MR
· (4) Auto-propagation to 14 consumer repos

Results

- **First-job bootstrap time cut from 5min to ~10s (~95% reduction)**, with 100% tool-version consistency across 14 consumer repos (15 repos total including the template provider)
- Compressed 12+ scattered version points into a 4-tier governance flow (ARG → central var → consumer indirection → lint drift check)
- Phase 4 cron now lands new versions of 5+ tools with just 2 manual clicks, effectively eliminating version-tracking overhead; the minor-guard policy prevents compatibility breakage in advance
- A risk-tiered auto-upgrade pipeline across 25 components (T1 stateless weekly / T2 platform biweekly / T3 stateful monthly + pre-flight) — version detection → per-component auto-MR → Slack alert → wave-sequenced apply, with a MAJOR_PIN guard and T3 atomic rollback for safety
- Built amd64 + arm64 multi-arch manifest lists via docker buildx (docker-container driver on a docker:dind sidecar), and mirrored external images (alpine·node·golang·rclone·awscli·docker) into Harbor with crane, removing external-registry dependency

Tech Stack

GitLab CI/CD, Docker Buildx (multi-arch), Docker BuildKit (docker-container driver), crane, Kaniko, Harbor, Helm / Helmfile, Bash / yq, Slack Webhook

---

Project 8: AWS EKS Production Game-Launch Platform (New Externally-Published Title)

Contribution 100% (Solo greenfield EKS design & build) 2026.06 ~ Present (Building & Operating)

Problem

Separately from the live game's estate (AWS QA·Prod), the new externally-published game ran only as a single on-premises development cluster, and launching it to production required a separate, scalable, highly-available AWS EKS production environment — on-premises alone could not handle global traffic and autoscaling needs.

Approach

Built a greenfield EKS cluster in eu-central-1, expanding the estate into an on-premises dev + AWS production multi-cluster hybrid, and managed ~11 platform components entirely via GitOps (ArgoCD).

Autoscaling / networking: Chose Karpenter over Cluster Autoscaler (unified spot across families, faster provisioning) to drive spot autoscaling across 9 spot instance families with a dedicated on-demand NodePool for game servers, while AWS Load Balancer Controller (ALB/NLB) + ExternalDNS (Route53) + a wildcard ACM cert automate ingress, DNS, and TLS.

Logging / secrets: an eck-operator-based EFK stack (Elasticsearch 3-node 3-AZ, EBS gp3, S3 snapshot SLM) runs HA, with External Secrets Operator externalizing secrets; every ServiceAccount authenticates via IRSA / Pod Identity, removing static keys.

Auth / deploy: ArgoCD integrates GitLab OAuth SSO + game-scoped RBAC; client artifacts ship via Jenkins → S3 + CloudFront, and server manifests reach the bastion's EFS mount via an S3 transit bucket + SSM SendCommand — the bastion is discovered dynamically by Name tag, so deploys use no SSH keys.

Troubleshooting

- Game-production Pods reserved far more memory (2Gi) than they actually used, causing node sprawl → right-sized memory requests to 512Mi from real usage, normalizing scale-out and improving spot cost efficiency
- During rolling deploys, ALB routed traffic to not-yet-ready Pods → combined ALB Pod readiness gate + PodDisruptionBudget + graceful drain to secure zero-downtime deploys

Results

- Expanded from a single on-premises dev cluster to an on-premises + AWS production multi-cluster hybrid, establishing the cloud production launch foundation for the new externally-published game

- Managed ~11 platform components 100% declaratively via GitOps (ArgoCD), establishing static-key-free authentication with IRSA / Pod Identity
- Keyless deployment via SSM SendCommand removed bastion SSH-key management overhead and key-leak risk
- ArgoCD app markers plus a production CI/CD transition doc make this a solo build the team can still operate and rebuild by the same procedure — a documented hand-off path

Tech Stack

AWS EKS (eu-central-1), Karpenter, AWS Load Balancer Controller, ExternalDNS, External Secrets Operator, ECK Operator / EFK, ArgoCD, IRSA / Pod Identity, AWS SSM (SendCommand), S3 / CloudFront, Kubernetes

---

NerdyStar

2023.03 ~ 2024.07

DevOps Engineer

Worked as a DevOps Engineer at a game and blockchain startup, leading the large-scale cloud migration project from AWS to GCP.

Project 1: Large-scale AWS → GCP Cloud Migration & CI/CD Build-out

Contribution: Infra design/migration lead 70% (remaining 30% = server-team application-side migration collab), CI/CD pipeline 100% 10 months (2023.03 ~ 2023.12)

Problem

Continuously rising AWS costs ($10,000+/month, high NAT Gateway and data transfer costs), a GCP Startup Program opportunity, and the need for GCP global load balancing for the multi-region game service

Approach

Adopted Shared VPC (Host / Service Project separation) and Workload Identity Federation so GitHub Actions could authenticate to GCP via short-lived tokens without service-account keys. Migrated DNS via subdomain delegation (Hosting.kr → AWS Route53 → GCP Cloud DNS) for pre-validation, then performed the final NS swap during a single scheduled maintenance window with minimal downtime.
Worked around Filestore's 2.5TB SSD minimum cost by self-hosting an NFS server on Compute Engine (pd-balanced 1TB). Configured Cloud Armor with region blocking + IP allow-list WAF policy, and split the Cloud CDN URL Map into HTTP / HTTPS variants to enforce HTTP → HTTPS redirect alongside static-asset caching.

Troubleshooting

- During AWS ALB → GCP Global LB transition, routing issues arose because AWS ALB still passes traffic to backends failing health checks whereas GCP LB blocks them.
  Reconfigured health checks/backends after analyzing GCP NEG structure
- GKE Ingress + BackendConfig health check returned 503 when the configured requestPath had no response.
  Resolved by guaranteeing the health-check endpoint response and aligning ManagedCertificate / FrontendConfig
- Game client file downloads failed after the CDN migration due to missing HTTP → HTTPS redirect.
  Resolved by splitting the URL Map into HTTP / HTTPS variants (`default_url_redirect.https_redirect=true`)
- After DNS delegation to Cloud DNS, the domain was missing from Google Search Console.
  Resolved by adding the issued verification TXT record to Cloud DNS to verify ownership, then re-registering

Results

- 50% cloud cost reduction ($10,000 → $5,000/month), 50% deployment time reduction, 95% rollback reduction
- Full GCP infrastructure Terraform IaC (TFLint/Checkov static analysis, Terratest unit testing), 100% deployment history tracking via Git commits
- Minimized downtime with resource-copy migration; final DNS cutover executed during an approximately 2-hour scheduled maintenance window aligned with game service operations
- Eliminated GCP service-account key management overhead and key-leak risk by adopting Workload Identity Federation for GitHub Actions

Tech Stack

GCP (GKE, Cloud SQL, Cloud Storage, VPC, Global LB 등), Shared VPC, Workload Identity Federation, Cloud Armor, Cloud CDN, Cloud DNS, NFS, Terraform, GitLab CI, ArgoCD, GitHub Actions, Kubernetes, Helm, Docker

Project 2: Game Data Collection Automation System

Contribution 100% 3 months (2024.01 ~ 2024.03)

Problem

Game and blockchain data was collected manually, costing analysts 2-3 hours daily, with frequent data loss caused by human error

Approach

Loaded multi-source data (MongoDB · CloudSQL · Google Analytics · Dune) into BigQuery — MongoDB via Dataflow ETL, CloudSQL via direct connection, GA/Dune via API — then triggered Daily / Monthly via Cloud Functions + Cloud Scheduler and pushed analyst-facing sheets via the Google Sheets API. A separate Cloud Function periodically deduplicated BigQuery data.
The entire infrastructure (datasets, Cloud Functions, schedulers, storage, Artifact Registry) was managed as Terraform IaC.

Results

- 95% data collection automation and 0% data gap rate by eliminating human error

Tech Stack

BigQuery, Dataflow, Cloud Functions, Cloud Scheduler, Cloud Storage, Artifact Registry, Terraform, Python, Google Sheets API, Docker

Project 3: PLG Stack Monitoring System

Contribution 100% 4 months (2024.04 ~ 2024.07)

Problem

No real-time monitoring system for Kubernetes clusters and applications, allowing only reactive responses with excessive time spent on root-cause analysis

Results

- SLI/SLO establishment and Alertmanager alarm noise optimization, proactive detection established, reduced response time, improved availability

Tech Stack

Prometheus, Loki, Grafana, Promtail, Fluent-bit, Slack API

Iorchard

2022.04 ~ 2023.03

Infra Engineer

Built PaaS (Kubernetes + OpenStack + Ceph) infrastructure solutions for clients and provided technical support, designing on-premises cloud infrastructure for financial and public-sector clients and leading operations training.

Key Projects: PaaS Solution · DP Training · Certificate Automation · Ansible Configuration Management

Results

- PaaS solution for 5 clients, stable operation on Kubernetes HA
- Established DP's autonomous Kubernetes operations (6-month training, avg 10 attendees · 4 hrs per session)
- 10x certificate renewal extension (1yr → 10yr), 50hrs annual ops savings
- Ansible-based server provisioning and configuration management automation eliminating configuration drift issues

Tech Stack

Kubernetes, OpenStack, Ceph, Ansible, kubeadm, Rocky Linux/CentOS

ETech System

2022.01 ~ 2022.04

System Engineer

Hardware Server and SAN storage construction. Transitioned to DevOps field for career growth.

Opensource Projects

Kubernetes CRD

k8s-namespace-sync — Automatically sync Kubernetes namespace resources across clusters
helios-lb — Custom load balancer controller for Kubernetes
network-policy-generator — Kubernetes NetworkPolicy auto-generation controller

GitHub Actions

Compress Decompress Action — Compress/decompress files in workflows
Image Tag Updater — Auto-update image tags in YAML/Helm files
Go Git Commit Action — Go-based Git commit creation and push
Extract Commit Action — Extract Git commit SHA, message, and author information

Multi Git Mirror Action — Automate mirroring across multiple Git repositories
Kube Diff Action — Compare and report Kubernetes resource changes
Major Tag Action — Auto-update major version tags to the latest semver release
Go Changelog Generator — Generate changelogs from Conventional Commits
Environment Output Setter — Set multiple env vars and workflow outputs with transformations
Ternary Operator Action — Conditional-logic action evaluating up to 10 conditions
Contributors Action — Auto-generate a contributors list from repo data
Kube Events Action — Check K8s cluster events with filtering and PR comments
Helm Kustomize Lint Action — Lint and validate Helm charts
Helm OCI Push — Push Helm charts to OCI registries

Ansible Galaxy

Ansible K8s IAC Tool — Automated K8s and IaC tool installation collection
Ansible User Management — Linux user and SSH key management role

DevOps Tools

bash-pilot — AI-powered Bash command assistant CLI tool
git-bridge — Git repository mirroring and sync CLI tool
static-file-server — Go-based static file server with directory UI, file preview, search, ZIP batch download
kube-diff — CLI to diff K8s manifests against a live cluster
kube-events — CLI to view and summarize K8s events

Chrome Extensions

Dev Toolkit — Developer utility collection
QRify — Instant QR code generation for URLs/text

Education

Chungnam National University - Public Administration (Transfer)	2018.03 ~ 2020.08
National Institute for Lifelong Education - Business Administration (Academic Credit Bank)	2016.09 ~ 2018.02

Certifications

Engineer Information Processing	2021.11
AWS Certified Solution Architect - Associate (Expired)	2021.11
Network Administrator Level 2	2021.10
Linux Master Level 2	2021.10
Computer Specialist Level 1	2021.04
TOEIC Speaking Test - 130/Intermediate Mid 3 (Expired)	2021.02

Study

Tech blog (somaz.blog, English) — DevOps · Kubernetes · IaC writing for global readers	2025.01 ~ 현재
Tech blog (somaz.tistory.com, Korean) — ongoing DevOps · Kubernetes · IaC learning and operations notes	2023.04 ~ 현재
T101(Terraform) Study	2023.08 ~ 2023.10
AEWS(EKS) Study	2023.04 ~ 2023.06
PKOS(Kubernetes) Study	2023.01 ~ 2023.02
Cloud Security Training	2022.02 ~ 2022.04
Security Solution Operation Specialist Training Program	2021.07 ~ 2021.12